

2.1 라즈베리 파이 개발 환경

■ 운영체제 설치

- 라즈베리 파이에서 가장 많이 사용하는 운영체제는 라즈비안
- 라즈비안은 리눅스 운영체제
- 리눅스
 - 리눅스는 1991년 리누스 토발스(Linus B. Torvalds)에 의해 개발
 - 개발자인 '리누스(Linus)'의 이름과 개발 모델로 삼은 '유닉스(UNIX)'의 이름을 합친 것
 - 리눅스는 소스가 공개되어 있는 오픈소스
 - 강력한 네트워킹 기능 제공
 - 멀티유저/멀티태스킹을 안정적으로 지원
 - 서버 및 PC 에서 비약적으로 빠르게 발전

2.1 라즈베리 파이 개발 환경

■ 운영체제 설치

- 마이크로 SD 메모리 카드에 라즈베리 파이 OS를 먼저 설치한 다음, 라즈베리 파이에 삽입하여 사용한다. 이를 위해 메모리 카드 리더를 이용하여 다른 PC에서 OS를 설치한다.
- 라즈베리 파이의 운영체제는 라즈비안(Raspbian)이다. 라즈비안은 공개 소프트웨어로 라즈베리 파이 홈페이지(<https://www.raspberrypi.com>)에서 다운로드 한다.

2.1 라즈베리 파이 개발 환경

■ 운영체제 설치

당사는 웹사이트에서 최상의 경험을 제공하기 위해 쿠키를 사용합니다. 이 사이트를 계속 방문하면 쿠키 사용에 동의하는 것입니다. 당사의 쿠키 정책을 읽으십시오.

알았어요!

 **Raspberry Pi**

가정용 산업용

하드웨어 **소프트웨어** 선적 서류 비치 소식 포럼 기반

1) 소프트웨어 선택

모두를 위한 컴퓨팅

은 산업에서 식탁 땀장이, 강의실 코더에 이르기까지 모든 사람이 컴퓨팅을 액세스 가능하고 저렴하게 만듭니다.

라즈베리 파이 구매

Raspberry Pi OS - Raspberry Pi x +

← → ↻ raspberrypi.com/software/

YouTube 지도 [태슬라] 부산 해운... 컴퓨터활용능력 2... IT관련 무료교육, 찾... NAVER 웹캠 사용법, 웹 카... 고속도로휴게소 전...

라즈베리 파이 OS

Raspberry Pi가 작동하려면 운영 체제가 필요합니다. 이거 야. Raspberry Pi OS(이전에는 Raspbian이라고 함)는 공식적으로 지원되는 운영 체제입니다.



Raspberry Pi Imager를 사용하여 Raspberry Pi OS 설치

Raspberry Pi Imager는 Raspberry Pi OS 및 기타 운영 체제를 microSD 카드에 설치하여 Raspberry Pi와 함께 사용할 수 있는 빠르고 쉬운 방법입니다. Raspberry Pi Imager를 사용하여 운영 체제를 설치하는 방법을 알아보려면 [45초 분량의 비디오를 시청하십시오.](#)

SD 카드 리더가 있는 컴퓨터에 Raspberry Pi Imager를 다운로드하여 설치합니다. Raspberry Pi와 함께 사용할 SD 카드를 리더기에 넣고 Raspberry Pi Imager를 실행합니다.

Windows 용 다운로드
[macOS용 다운로드](#)
[x86용 우분투 다운로드](#)



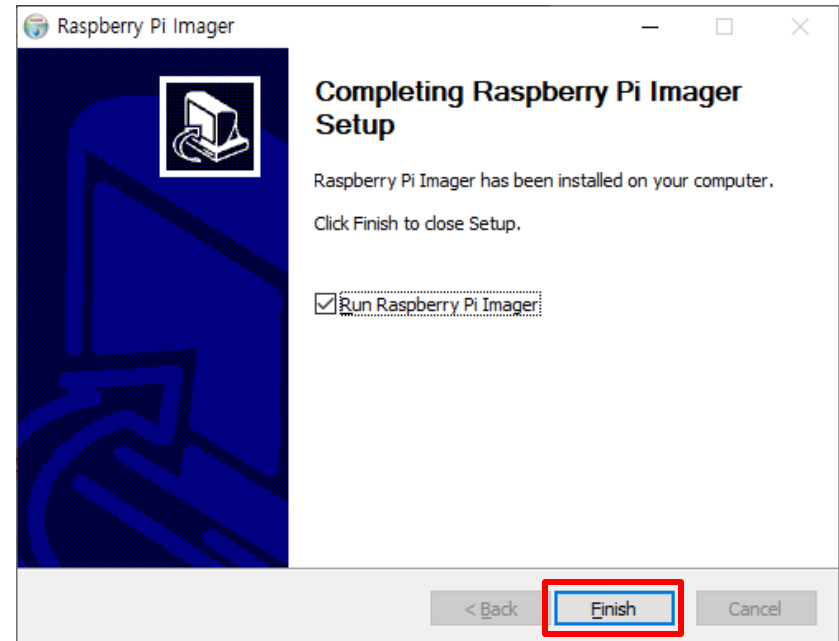
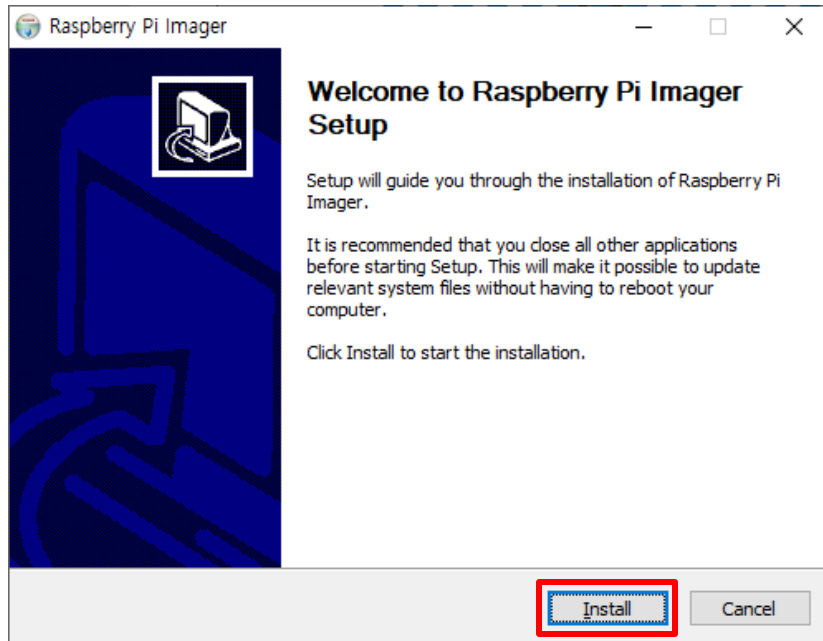
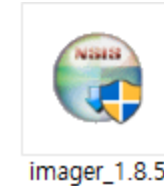
2) 윈도우용 다운로드 후 설치

Page 3

2.1 라즈베리 파이 개발 환경

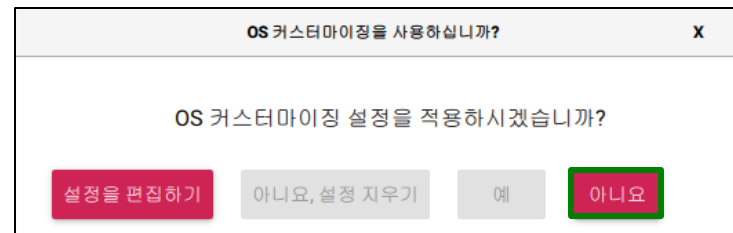
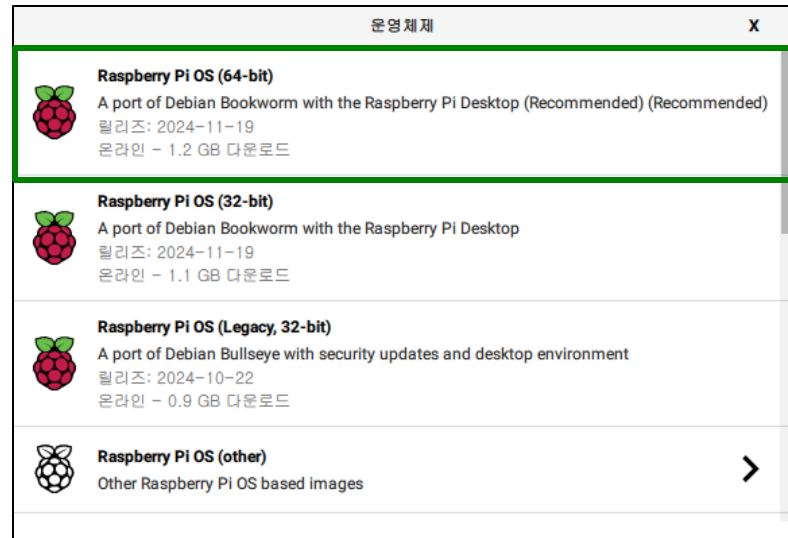
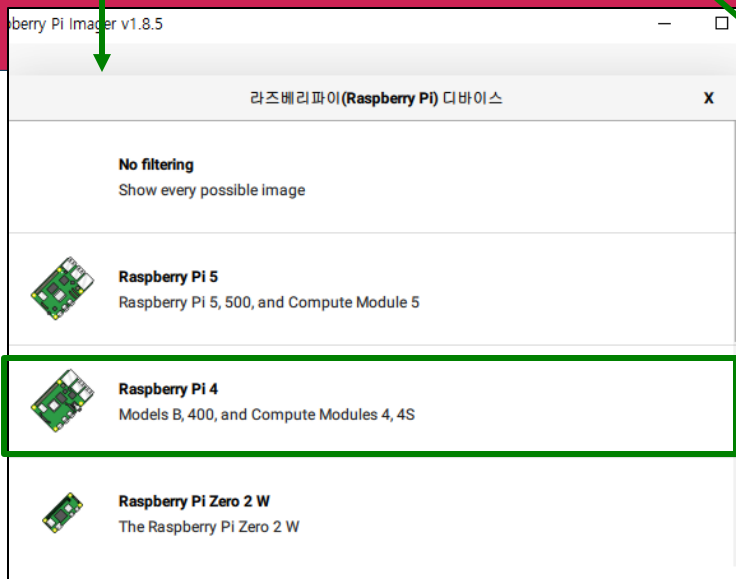
■ 운영체제 설치

- 설치 파일을 다운로드(imager_1.8.5) 후 실행



2.1 라즈베리 파이 개발 환경

■ 운영체제 설치



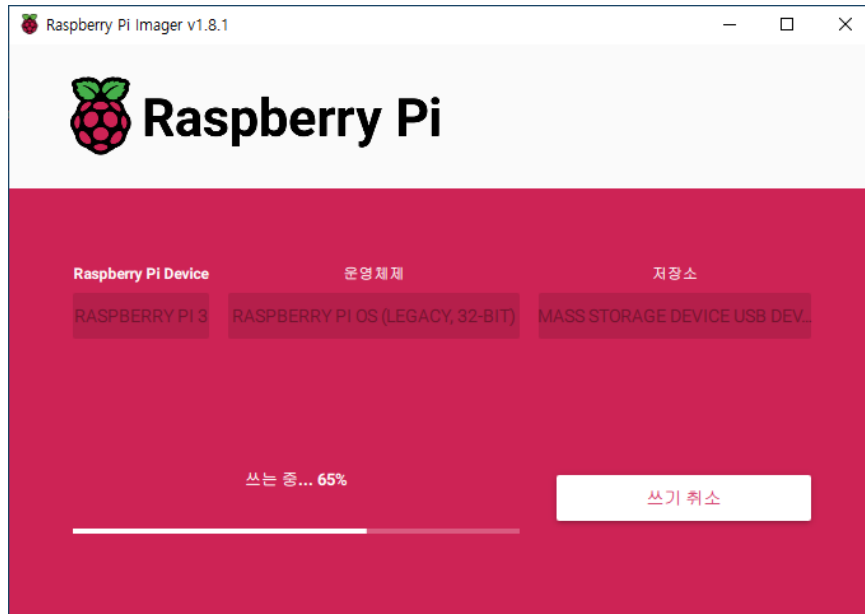
2.1 라즈베리 파이 개발 환경

■ 운영체제 설치



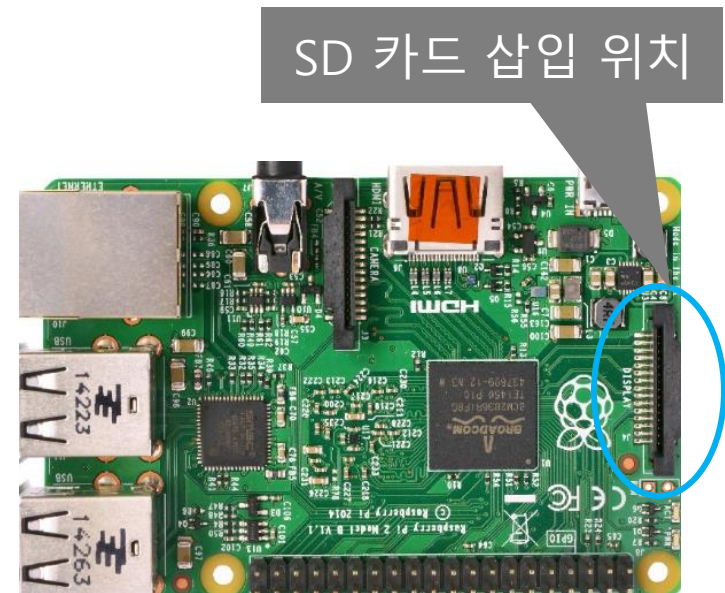
2.1 라즈베리 파이 개발 환경

■ 운영체제 설치



2.1 라즈베리 파이 개발 환경

■ 운영체제 설치



- 쓰기 완료 후 SD 메모리를 라즈베리 파이에 삽입

2.1 라즈베리 파이 개발 환경

■ 운영체제 설치

- 라즈비안의 부팅 진행
- Welcome to the Raspberry Pi Desktop [Next]
- Set Country
 - [Country :]
 - [Language :]
 - [Timezone :] [Next]
- Create User
 - [Enter username : xxxx]
 - [Enter password : yyyy]
 - [Confirm password : yyyy] [Next]

2.1 라즈베리 파이 개발 환경

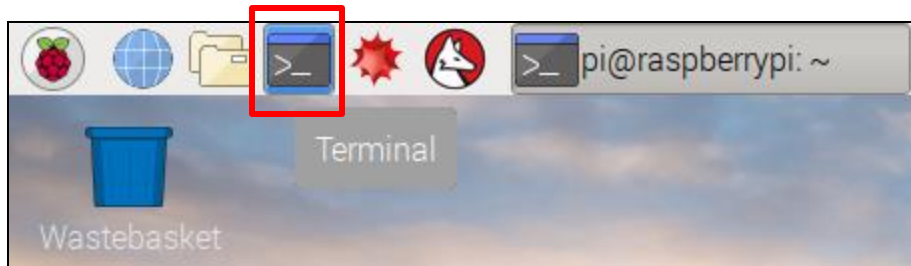
■ 운영체제 설치

- Set up Screen [Next]
- Select WiFi Network
[NETGEAR40] [Next]
[Password : i423i423] [Next]
- Choose Browser [Next]
- Update Software
[Next] ➔ 업데이트 진행 ➔ [OK]
- Setup completed
[Restart]

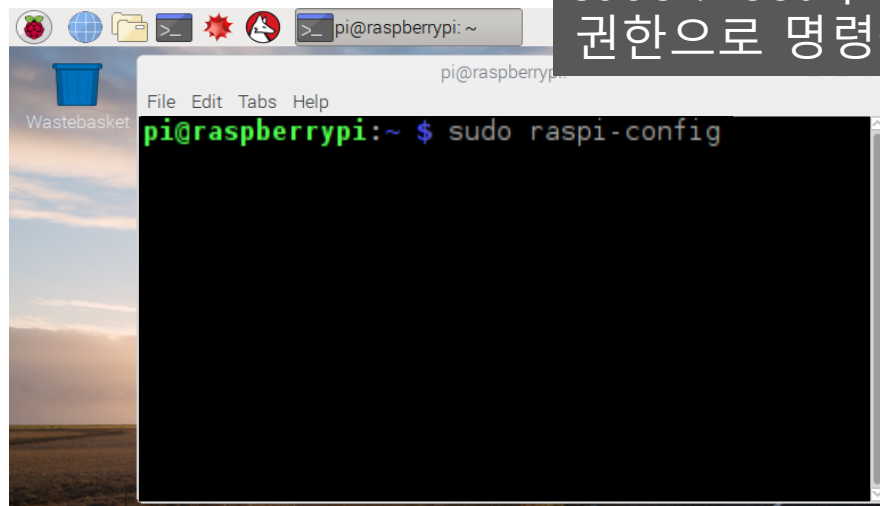
2.1 라즈베리 파이 개발 환경

■ 운영체제 설치

- 부팅 후 바탕화면의 왼쪽 상단 아이콘 중 Terminal 클릭



- `>sudo raspi-config` 입력



2.1 라즈베리 파이 개발 환경

■ 라즈비안 설정 - 파일시스템

- "6 Advanced Options"에서 설정이 가능한 "A1 Expand Filesystem"을 설정한다. 초기 파일 시스템을 SD 메모리 카드의 최대 크기까지 확장시켜 준다. 특정 버전에서는 파일 시스템이 자동으로 확장이 되지만 자동으로 안되는 버전을 위해 설정을 진행한다.
- Root partition has been resized [Enter]

```
1. System Options
2. Display Options
3. Interface Options
4. Performance Options
5. Localisation Options
6. Advanced Options
8. Update
9. About raspi-config
```

2.1 라즈베리 파이 개발 환경

■ 라즈비안 설정 – SPI(Serial Peripheral Interface)설정

- “3 Interface Options”을 선택하여 SPI 설정을 변경한다.

1. System Options
2. Display Options
3. Interface Options
4. Performance Options
5. Localisation Options
6. Advance Options
8. Update
9. About raspi-config

** 직렬통신 방식 중 하나로 주변장치와 데이터 통신을 위한 기능*

2.1 라즈베리 파이 개발 환경

■ 라즈비안 설정 – SPI 설정

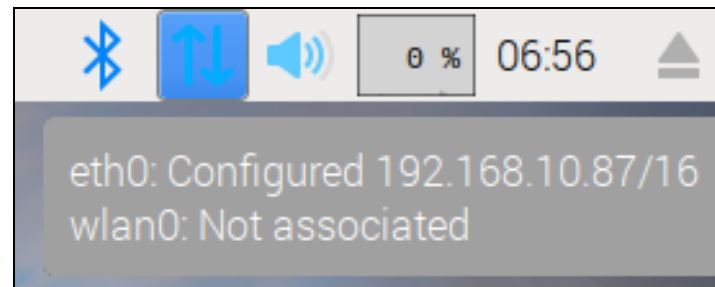
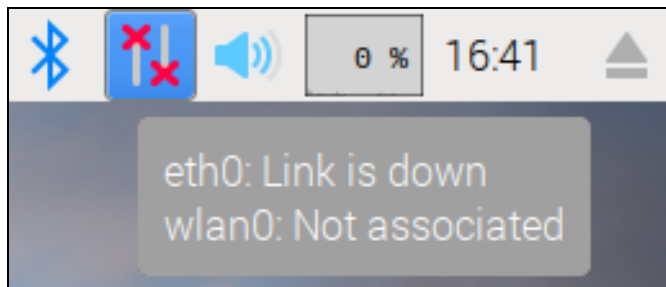
- “I4 SPI”를 선택한다.
- SPI 인터페이스를 사용하기 위해 “예”를 선택한다.
- [Enter]
- 설정이 끝나면 “Tab”키를 눌러 <Finish>를 선택하고, 재부팅 할지 여부를 선택하는 화면이 나오면, <YES>를 선택하여 재부팅하거나 다음과 같이 “reboot” 명령을 사용하여 재부팅한다.

```
pi@raspberrypi:~ $ reboot
```

2.1 라즈베리 파이 개발 환경

■ 네트워크 설정 - 유선네트워크

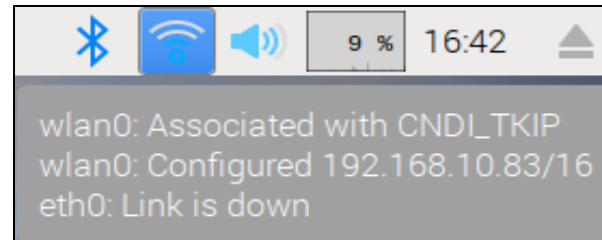
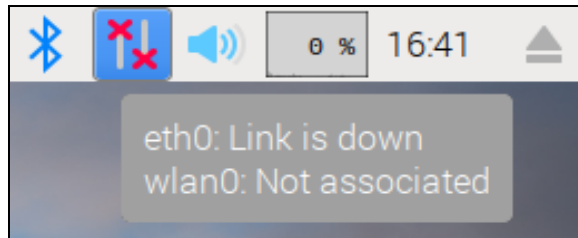
- LAN 케이블을 이용하는 경우, 라즈베리 파이 보드의 LAN 포트에 연결을 해주면 자동으로 IP를 할당 받아 설정한다. 이때, 네트워크 설정이 정상적으로 이루어졌는지 오른쪽 상단에 위치한 네트워크 설정 아이콘을 통해 확인 가능하다. 이 아이콘 위에 마우스 포인터를 올려놓으면 할당된 IP가 표시된다.



2.1 라즈베리 파이 개발 환경

■ 네트워크 설정 - 와이파이

- 스피커 아이콘 옆에 위치한 네트워크 설정 아이콘을 클릭하면, 라즈베리 파이에 내장된 Wi-Fi를 통하여 인식된 AP 목록이 표시된다. 사용하고자 하는 AP를 선택하여 설정되어 있는 암호를 입력한다. 다시 네트워크 설정 아이콘 위에 마우스 포인터를 위치하면, 할당된 IP 주소를 확인 할 수 있다



2.1 라즈베리 파이 개발 환경

- 업그레이드 해야 할 새로운 버전이 있는지 확인

>sudo apt-get update

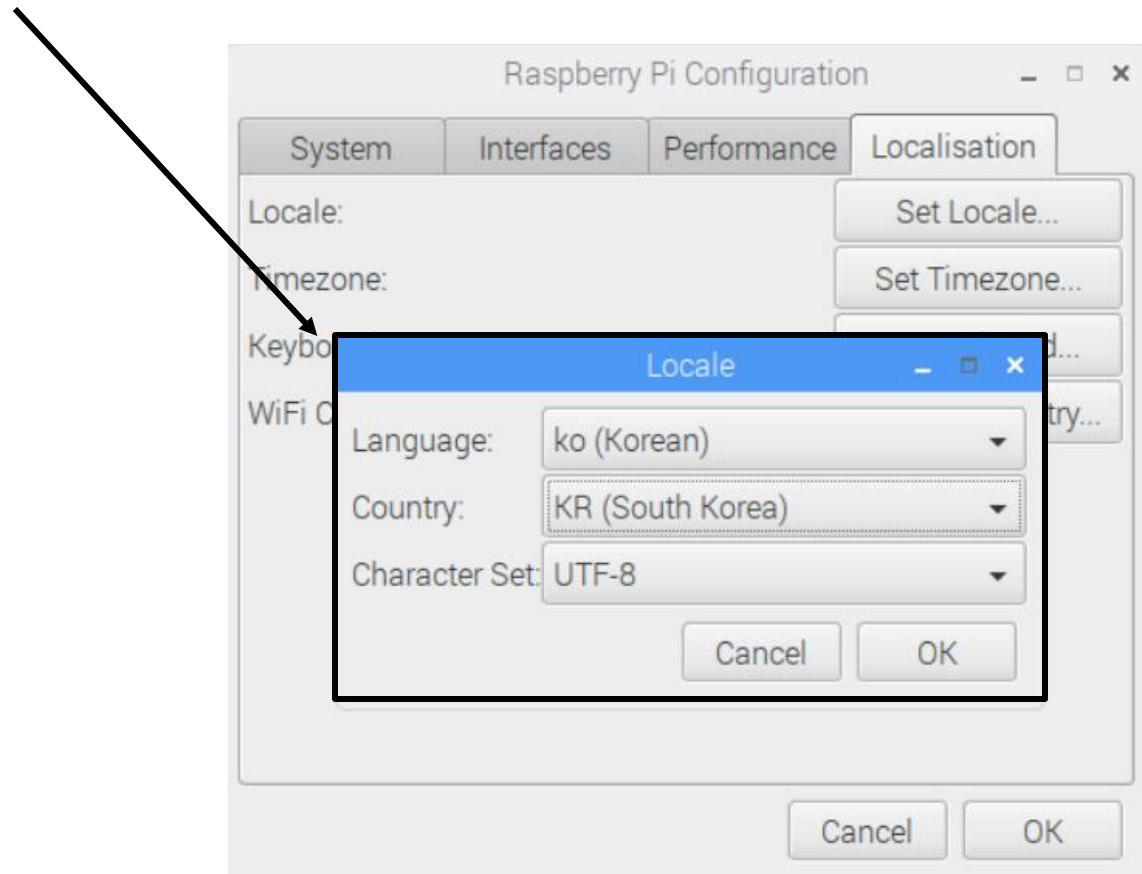
- 새로운 버전으로 업그레이드 진행

>sudo apt-get upgrade

2.1 라즈베리 파이 개발 환경

■ 한글

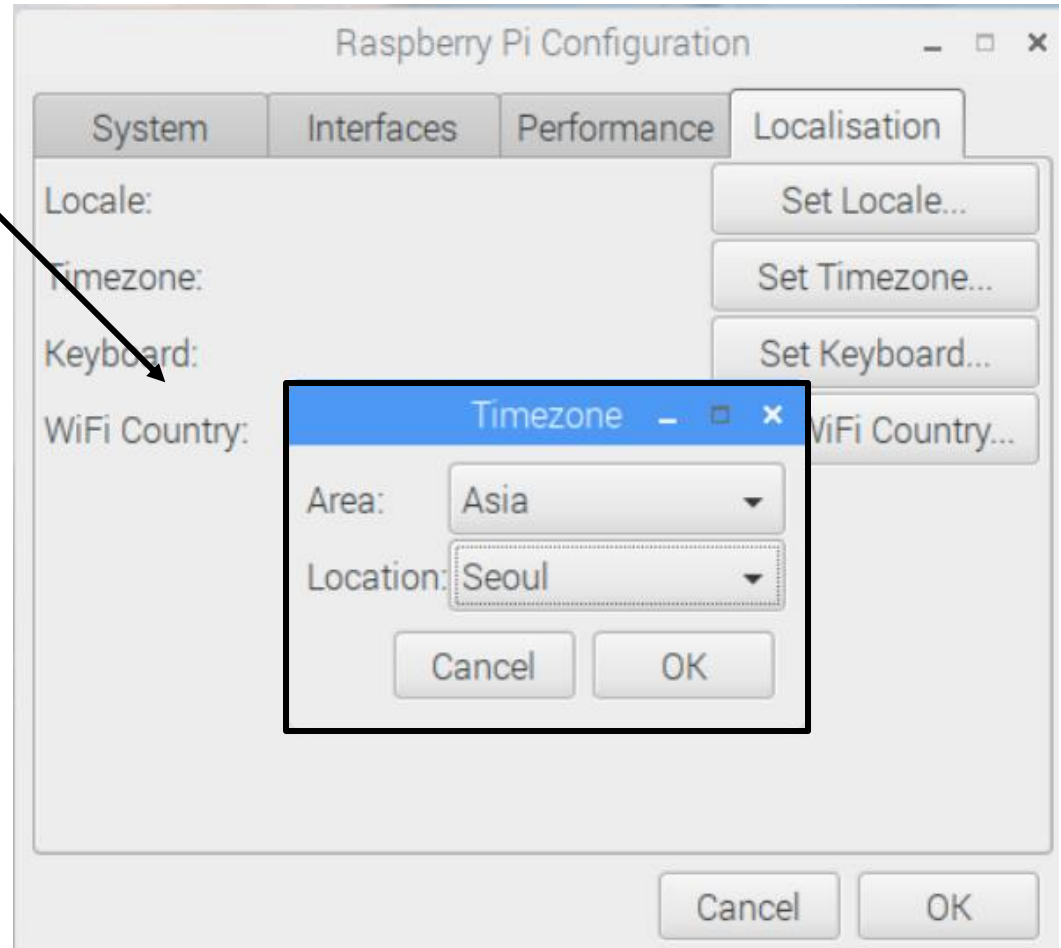
- 라즈베리 파이 로고 ➔ Preferences ➔ Raspberry Pi Configuration
Localisation-set Locale



2.1 라즈베리 파이 개발 환경

■ 한글

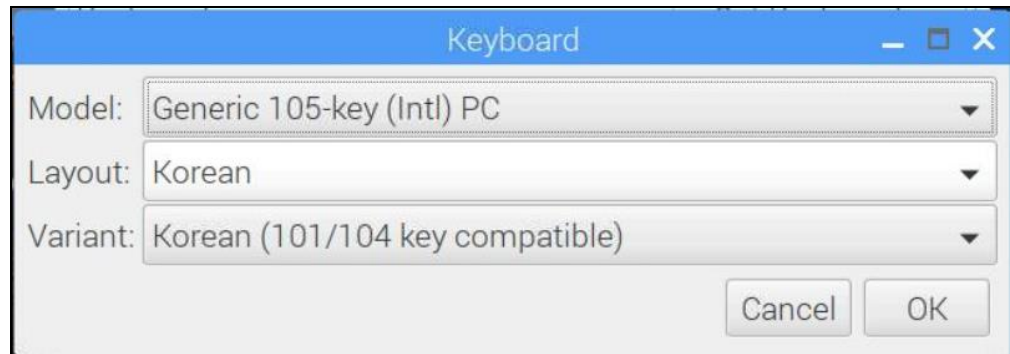
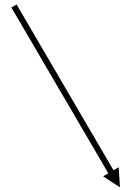
- Localisation-set Timezone



2.1 라즈베리 파이 개발 환경

■ 한글

Localisation-set Keyboard



2.1 라즈베리 파이 개발 환경

■ 한글

- 인터넷 접속을 통하여 한글 디스플레이 확인, 만약 한글이 깨어져 표시 되는 경우

>sudo apt-get install ibus //ibus는 다국어 입력기.....[Y/n]

>sudo apt-get install ibus-hangul // 한글 입력기

>sudo apt-get install fonts-unfonts-core //한글폰트

>ibus engine hangul //ibus의 입력 엔진을 한국어로 설정

>sudo reboot

오류시 재부팅 후 다시 진행!

*sudo(Super User Do)의 줄임 말로 root 권한으로 실행

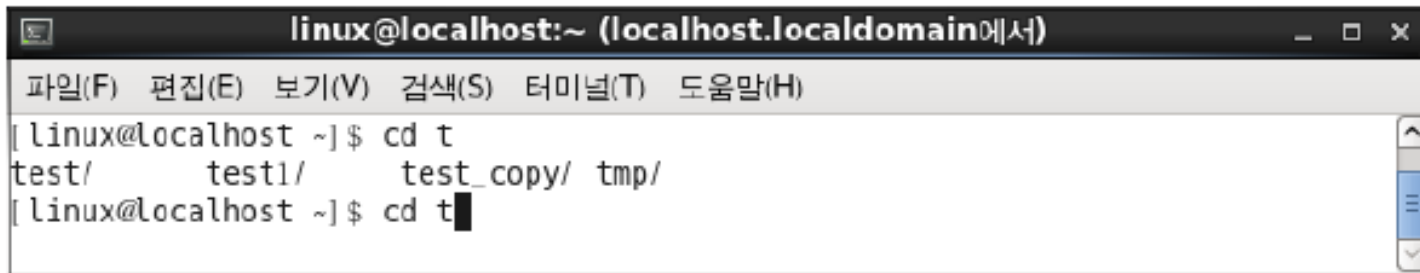
2.1 라즈베리 파이 개발 환경

■ reboot

- 시스템 재시작

■ 문자열 자동완성

- 명령어 입력시 파일명이나 디렉토리명을 인수로 쓰게 되는 경우 tab 키를 이용하여 문자열을 자동완성해주는 기능

A terminal window titled 'linux@localhost:~ (localhost.localdomain에서)' with a menu bar containing '파일(F)', '편집(E)', '보기(V)', '검색(S)', '터미널(T)', and '도움말(H)'. The terminal shows the command '[linux@localhost ~]\$ cd t' followed by a list of suggestions: 'test/' (highlighted), 'test1/', and 'test_copy/ tmp/'. The second line shows '[linux@localhost ~]\$ cd t' with the cursor at the end of the command.

```
linux@localhost:~ (localhost.localdomain에서)
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
[linux@localhost ~]$ cd t
test/      test1/     test_copy/ tmp/
[linux@localhost ~]$ cd t
```

2.1 라즈베리 파이 개발 환경

■ 라즈비안 설정

- 라즈비안의 셸에서

>sudo raspi-config

```
pi@raspberrypi:~ $ sudo raspi-config
```

- Raspberry Pi Software Configuration Tool (raspi-config) 화면이 표시되고 몇 가지 설정을 할 수 있다.

명령어	의 미
sudo	사용자로 로그인 한 후 슈퍼유저 권한의 명령어를 실행

2.1 라즈베리 파이 개발 환경

■ 시스템 종료

라즈베리파이 로그인 ➔ 로그아웃 ➔ Shutdown

■ 시스템 재시작

라즈베리파이 로그인 ➔ 로그아웃 ➔ Reboot

■ 사용자 로그인

라즈베리파이 로그인 ➔ 로그아웃 ➔ Logout

2.1 라즈베리 파이 개발 환경

■ GPIO 소개

- GPIO(General Purpose Input Output)란, 범용으로 사용되는 입출력 포트를 말하며, 설계자가 원하는 동작을 수행하기 위하여 포트를 입력 및 출력으로 구성하여 신호를 전달한다.
- 간단히 예를 들면, LED ON/OFF, FND 제어, Buzzer 소리를 켜기 위해서 해당 소자에 0 혹은, 1의 신호를 보내주는 것이다.
- 라즈베리 파이 보드 상에 40개의 핀들이 외부와 인터페이스가 가능하도록 구성되어 있다.

2.1 라즈베리 파이 개발 환경

■ GPIO 소개



Raspberry Pi 3 GPIO Header

Pin#	NAME		NAME	Pin#
01	3.3v DC Power	⬤ ⬤	DC Power 5v	02
03	GPIO02 (SDA1 , I ² C)	⬤ ⬤	DC Power 5v	04
05	GPIO03 (SCL1 , I ² C)	⬤ ⬤	Ground	06
07	GPIO04 (GPIO_GCLK)	⬤ ⬤	(TXD0) GPIO14	08
09	Ground	⬤ ⬤	(RXD0) GPIO15	10
11	GPIO17 (GPIO_GEN0)	⬤ ⬤	(GPIO_GEN1) GPIO18	12
13	GPIO27 (GPIO_GEN2)	⬤ ⬤	Ground	14
15	GPIO22 (GPIO_GEN3)	⬤ ⬤	(GPIO_GEN4) GPIO23	16
17	3.3v DC Power	⬤ ⬤	(GPIO_GEN5) GPIO24	18
19	GPIO10 (SPI_MOSI)	⬤ ⬤	Ground	20
21	GPIO09 (SPI_MISO)	⬤ ⬤	(GPIO_GEN6) GPIO25	22
23	GPIO11 (SPI_CLK)	⬤ ⬤	(SPI_CE0_N) GPIO08	24
25	Ground	⬤ ⬤	(SPI_CE1_N) GPIO07	26
27	ID_SD (I ² C ID EEPROM)	⬤ ⬤	(I ² C ID EEPROM) ID_SC	28
29	GPIO05	⬤ ⬤	Ground	30
31	GPIO06	⬤ ⬤	GPIO12	32
33	GPIO13	⬤ ⬤	Ground	34
35	GPIO19	⬤ ⬤	GPIO16	36
37	GPIO26	⬤ ⬤	GPIO20	38
39	Ground	⬤ ⬤	GPIO21	40

2.1 라즈베리 파이 개발 환경

■ GPIO 라이브러리

- 라즈베리 파이의 GPIO를 제어하기 위해서 사용되는 언어는 C, C++, Java, Python 등 매우 다양하며, 각 언어에 따라 사용자들이 라이브러리를 작성하여 공유하고 있다. (C 사용)
- wiringPi는 GPIO 접근 라이브러리이며, 아두이노처럼 pinMode(pin, OUTPUT) 함수를 사용하여 입출력을 설정할 수 있다.

2.1 라즈베리 파이 개발 환경

■ WiringPi 설치

```
>sudo apt-get install git-core //git 설치
>git clone https://github.com/WiringPi/WiringPi
>cd WiringPi //디렉토리 이동
>./build
```

- WiringPi는 분산 버전 관리시스템 GIT에 의해 관리되므로 먼저, git 설치
- WiringPi 라이브러리 다운로드
- WiringPi 디렉토리로 이동하여 WiringPi 라이브러리를 build

```
pi@raspberrypi:~ $ sudo apt-get install git-core
pi@raspberrypi:~ $ git clone https://github.com/WiringPi/WiringPi
Cloning into 'WiringPi'...
remote: Enumerating objects: 2557, done.
remote: Counting objects: 100% (1190/1190), done.
remote: Compressing objects: 100% (1190/1190), done.
remote: Total 2557 (delta 1024), reused 1190, compressed 1190, copying 1367 objects, done.
Receiving objects: 100% (2557/2557), 1.1 MiB | 1.1 MiB/s, done.
Resolving deltas: 100% (1718/1718), done.
pi@raspberrypi:~/WiringPi $ cd wiringPi/
pi@raspberrypi:~/WiringPi/wiringPi $ ls
README.TXT  build  debian  examples  pins
pi@raspberrypi:~/WiringPi/wiringPi $ ./build
```

2.1 라즈베리 파이 개발 환경

■ WiringPi 설치

- 빌드 완료

```
[Install]
All Done.

NOTE: To compile programs with wiringPi, you need to add:
  -lwiringPi
to your compile line(s) To use the Gertboard, MaxDetect, etc.
code (the devLib), you need to also add:
  -lwiringPiDev
to your compile line(s).

pi@raspberrypi ~/wiringPi $
```

>gpio -v //설치 확인

```
pi@raspberrypi:~ $ gpio -v
gpio version: 3.10
Copyright (c) 2012-2024 Gordon Henderson and contributors
This is free software with ABSOLUTELY NO WARRANTY.
For details type: gpio -warranty

Hardware details:
  Type: Pi 4B, Revision: 05, Memory: 2048MB, Maker: Sony

System details:
  * Device tree present.
    Model: Raspberry Pi 4 Model B Rev 1.5
  * Supports full user-level GPIO access via memory.
  * Supports basic user-level GPIO access via /dev/gpiomem.
  * Supports basic user-level GPIO access via /dev/gpiochip (slow).
```

2.1 라즈베리 파이 개발 환경

■ WiringPi 설치

```
>gpio -v //설치 확인
>gpio readall //핀맵 확인
```

```
pi@raspberrypi:~ $ gpio -v
gpio version: 3.10
Copyright (c) 2012-2024 Gordon Henderson and contributors
This is free software with ABSOLUTELY NO WARRANTY.
For details type: gpio -warranty

Hardware details:
  Type: Pi 4B, Revision: 05, Memory: 2048MB, Maker: Sony

System details:
  * Device tree present.
    Model: Raspberry Pi 4 Model B Rev 1.5
  * Supports full user-level GPIO access via memory.
```

```
pi@raspberrypi:~/wiringPi $ gpio readall
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| BCM | wPi |   Name   | Mode | V | Physical | V | Mode |   Name   | wPi | BCM |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|     |     | 3.3v     |      |   | 1 || 2 |     |     | 5v       |     |     |
| 2   | 8   | SDA.1    | IN    | 1 | 3 || 4 |     |     | 5V       |     |     |
| 3   | 9   | SCL.1    | IN    | 1 | 5 || 6 |     |     | 0v       |     |     |
| 4   | 7   | GPIO. 7  | IN    | 1 | 7 || 8 | 0 | IN  | TxD      | 15  | 14  |
|     |     | 0v       |      |   | 9 || 10| 1 | IN  | RxD      | 16  | 15  |
| 17  | 0   | GPIO. 0  | IN    | 1 | 11|| 12| 1 | IN  | GPIO. 1  | 1   | 18  |
| 27  | 2   | GPIO. 2  | IN    | 1 | 13|| 14|   |     | 0v       |     |     |
| 22  | 3   | GPIO. 3  | IN    | 1 | 15|| 16| 1 | IN  | GPIO. 4  | 4   | 23  |
|     |     | 3.3v     |      |   | 17|| 18| 1 | IN  | GPIO. 5  | 5   | 24  |
| 10  | 12  | MOSI     | ALT0  | 1 | 19|| 20|   |     | 0v       |     |     |
| 9   | 13  | MISO     | ALT0  | 1 | 21|| 22| 1 | IN  | GPIO. 6  | 6   | 25  |
| 11  | 14  | SCLK     | ALT0  | 0 | 23|| 24| 1 | OUT | CE0      | 10  | 8   |
|     |     | 0v       |      |   | 25|| 26| 1 | OUT | CE1      | 11  | 7   |
| 0   | 30  | SDA.0    | IN    | 1 | 27|| 28| 1 | IN  | SCL.0    | 31  | 1   |
| 5   | 21  | GPIO.21  | IN    | 1 | 29|| 30|   |     | 0v       |     |     |
| 6   | 22  | GPIO.22  | IN    | 1 | 31|| 32| 1 | IN  | GPIO.26  | 26  | 12  |
| 13  | 23  | GPIO.23  | IN    | 1 | 33|| 34|   |     | 0v       |     |     |
| 19  | 24  | GPIO.24  | IN    | 0 | 35|| 36| 1 | IN  | GPIO.27  | 27  | 16  |
| 26  | 25  | GPIO.25  | IN    | 0 | 37|| 38| 0 | IN  | GPIO.28  | 28  | 20  |
|     |     | 0v       |      |   | 39|| 40| 0 | IN  | GPIO.29  | 29  | 21  |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| BCM | wPi |   Name   | Mode | V | Physical | V | Mode |   Name   | wPi | BCM |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
pi@raspberrypi:~/wiringPi $
```

2.2 라즈베리 파이 프로그래밍

■ 프로그램 테스트

- vi 편집기 : 리눅스에서 가장 많이 사용하는 텍스트 편집기로서 리눅스 계열의 대부분의 운영체제에서 광범위하게 사용
- 처음에는 사용에 어려움이 있으나 숙달되면 마우스 입력 없이 작업이 가능해 매우 빠르게 편집 가능한 장점이 있음
- 입력 모드 : 텍스트의 입력/삭제
- 편집 모드 : 복사, 자르기, 붙이기 수행
- 명령 모드 : 파일을 저장, 종료

2.2 라즈베리 파이 프로그래밍

■ 소스코드

>vi helloworld.c

Program Source	Description
#include <stdio.h>	표준 입출력 헤더파일 포함
int main (void)	메인 함수
{	
printf("Hello World\n");	"Hello World" 문장을 모니터에 출력
}	

2.2 라즈베리 파이 프로그래밍

■ 소스코드

- shift zz 로 프로그램 종료 및 프로그램 저장
- 프로그램 빌드(Build)
 >gcc -o hello helloworld.c
- 프로그램 실행
 >./hello

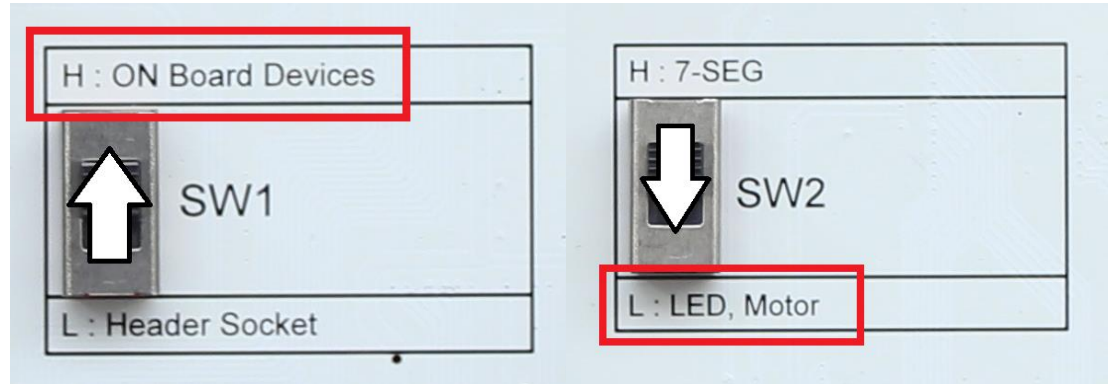
2.3 디바이스 제어(LED)

■ 스위치 설정

- SW1 스위치를 H : ON Board Devices 방향으로 위치
- SW2 스위치를 L : LED, Motor 방향으로 위치

■ LED On/Off 출력

- LED와 연결된 GPIO 단자에 High를 출력하면 LED on
- LED와 연결된 GPIO 단자에 Low를 출력하면 LED off



2.3 디바이스 제어(LED)

■ LED 제어

> vi led.c ↵

✓wiringPi에서 사용하는 Delay는 2가지가 있다.
millisecond단위의 delay와 microseconds 단위의 delayMicroseconds

```
#include <wiringPi.h>

int LED_1=4;

int main (void)
{
    wiringPiSetupGpio();
    pinMode(LED_1, OUTPUT);
    digitalWrite(LED_1, LOW);
```

```
while(1)
{
    digitalWrite(LED_1, HIGH); //ON
    delay(500);
    digitalWrite(LED_1, LOW); //OFF
    delay(500);
} //while

} //main
```

2.3 디바이스 제어(LED)

- 프로그램을 컴파일하고, 생성된 파일을 실행하여 동작을 확인한다.
 - 컴파일 명령 : `gcc -o led led.c -lwiringPi`
 - 실행 명령 : `sudo ./led`
 - wiringPi 라이브러리를 사용한 경우 컴파일 명령의 끝에 `-lwiringPi` 추가
- Ctrl+C를 눌러 프로그램을 종료한다.
- **라즈베리파이 사용 종료 시, Shutdown 하고 전원을 off 한다.**